...and in let declarations:

08-metadata/10/spec/music_storage_spec.rb
```
RSpec.describe 'Music storage' do
  let(:s3_client) do |example|
    S3Client.for(example.metadata[:s3_adapter])
  end

  it 'stores music on the real S3', s3_adapter: :real do
    # ...
  end

  it 'stores music on an in-memory S3', s3_adapter: :memory do
    # ...
  end
end
```

These specs both rely on an S3 client, but each uses a different one. One spec will run with S3Client.for(:real), and the other with S3Client.for(:memory).

So far, we've seen how to read what metadata is defined on an example, and how to write metadata on one or more examples. The real fun starts when we fire up RSpec and actually use all this information to change its behavior.

Selecting Which Specs to Run

When you're running your specs, you often want to change *which ones* you include. In this section, we're going to show you a few different situations where this kind of slicing and dicing comes in handy.

Filtering

Most of the time when we start RSpec, we don't run the entire suite. We're either running unit specs for a specific class we're designing or we're kicking off some integration specs to catch regressions.

We first touched on running specific examples in *Running Just What You Need*, on page 20. You saw how to select examples by pass/fail status, by our need to focus on them, and by custom tags. Let's take a deeper dive into how to control which specs to run.

Excluding Examples

Sometimes, you want to exclude a set of examples from your RSpec run. For instance, when you're testing a project for compatibility across Ruby interpreters, you may have a few examples that are implementation-specific. You can give these a tag such as :jruby_only, and then use your RSpec.configure block to omit those specs when you're testing on another Ruby interpreter: